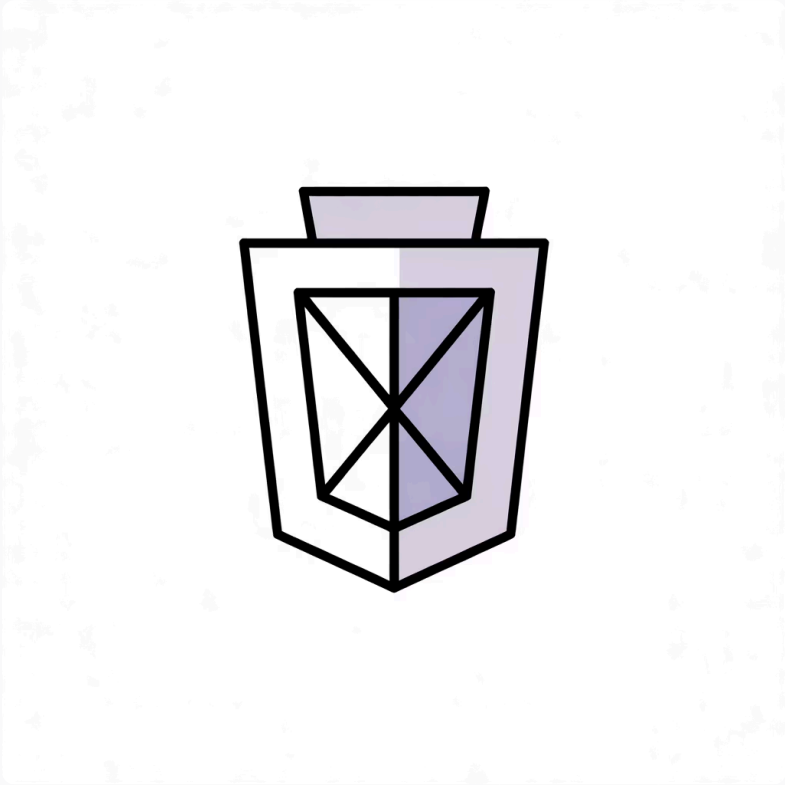
JavaScript - Jour 9

Découverte de TypeScript et de la Canvas API

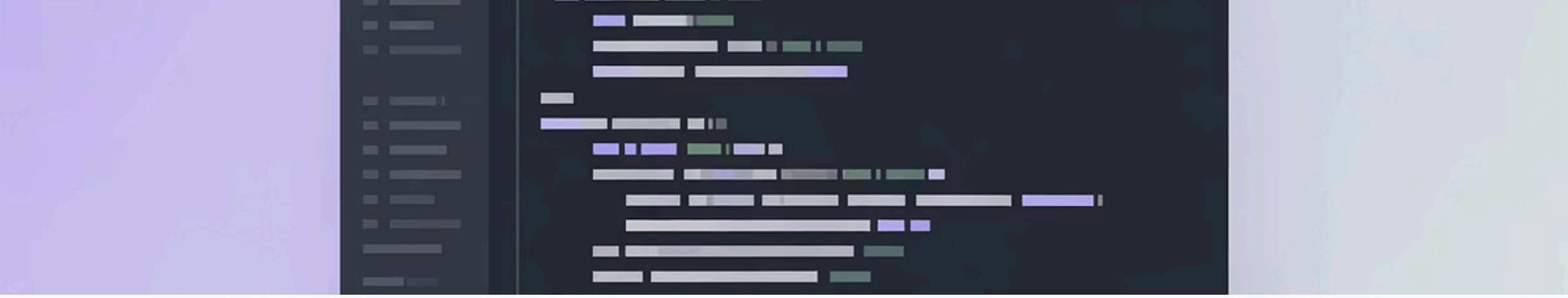


Objectifs pédagogiques de la journée

- Introduction à TypeScript
- Introduction à Canvas API

Avant de continuer…

On va se créer un tout dernier projet avec Vite !



Même chose que pour le MVC, mais avec TypeScript !

La dernière fois (qui était aussi la première fois), on avait choisi "JavaScript" dans le formulaire de Vite.
Cette fois-ci : on va expressément choisir **TypeScript** à la place.

Étapes

1

Ouvrir un terminal (Powershell ou Bash/ZSH)

Placez-vous ensuite dans le répertoire dans lequel vous voulez créer le nouveau projet.

2

Exécutez la commande suivante dans le terminal

`npm create vite`

Remplissez ensuite le questionnaire dans le terminal avec les paramètres souhaités.

3

Ouvrez votre éditeur de code dans le dossier du projet

`code vite-project` (VS Code) ou `webstorm vite-project` (WebStorm)

4

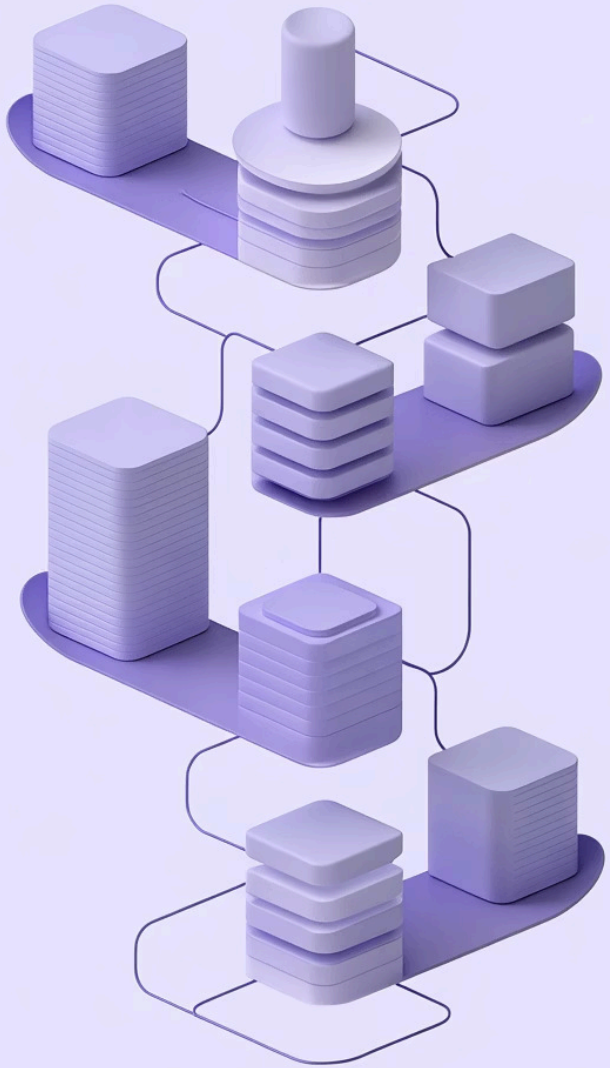
Installez les dépendances du projet

À la racine de votre projet (obligatoire), lancez la commande `npm install` pour installer toutes les dépendances nécessaires pour le projet.

5

Exécutez le projet

`npm run dev` lancera le projet et ouvrira un serveur sur votre machine pour pouvoir y accéder dans un navigateur.



MODULE

Introduction à TypeScript

Introduction à TypeScript

Ce que nous allons explorer

- Présentation générale du langage TypeScript
- Les principales distinctions avec JavaScript
- Les bénéfices du typage statique
- Le processus de compilation (transpilation)

Les concepts fondamentaux

TypeScript = JavaScript + Types

Une surcouche de JavaScript qui ajoute des types.

Compilation vers JavaScript

Convertit le code TS en JS standard pour l'exécution.

Détection d'erreurs anticipée

Identifie les bugs au développement, avant l'exécution.

Autocomplétion et documentation

Améliore l'expérience de développement et la lisibilité.

Code plus maintenable

Facilite la compréhension et la gestion des projets complexes.

> « TypeScript, c’est du JavaScript moins ”pâte-à-modeler teuteu”. »
>
> — Gauthier Daniels, janvier 2026

> « Ce n’est pas compliqué, mais c’est complexe. »
>
> — Même énergumène, même mois

Comparaison basique JS vs TS

JavaScript (extension : fichier.js)

```
// Erreur détectée seulement à l'exécution
function calculerPrix(quantite, prixUnitaire) {
  return quantite * prixUnitaire;
}

calculerPrix("5", 10); // "50" au lieu de 50 (erreur silencieuse)
calculerPrix(5);       // NaN (oubli de paramètre, soit number * undefined === NaN)
calculerPrix(5, 10);   // 50
```

TypeScript (extension : fichier.ts)

```
// TypeScript - Erreur détectée à la compilation (même avant, via l'éditeur !)
function calculerPrix(quantite: number, prixUnitaire: number): number {
  return quantite * prixUnitaire;
}

calculerPrix("5", 10); // Erreur : le type 'string' ne peut pas être assigné au type 'number'
calculerPrix(5);       // Erreur : argument manquant
calculerPrix(5, 10);   // 50
```

Types primitifs et types TypeScript

`number`

Utilisé pour représenter les nombres, qu'ils soient entiers ou à virgule flottante.

`string`

Désigne les chaînes de caractères, le texte.

`boolean`

Représente une valeur logique : `true` ou `false`.

`null`

Indique l'absence intentionnelle d'une valeur.

`undefined`

Signifie qu'une variable a été déclarée mais n'a pas encore reçu de valeur.

Types personnalisés

Au-delà des types primitifs, TypeScript permet de définir des types personnalisés pour mieux structurer et valider les données de vos applications.

`type` (Alias de type)

Donnez un nouveau nom à un type existant ou combinez-en plusieurs pour plus de clarté et de réutilisabilité.

`enum` (Énumération)

Définissez un ensemble de constantes nommées, améliorant la lisibilité et réduisant les erreurs liées aux valeurs arbitraires.

`interface` (Interface)

Décrivez la forme des objets pour garantir la cohérence des structures de données à travers votre code.

`Record` (Type utilitaire)

Créez des objets où toutes les clés et toutes les valeurs suivent un type spécifique, idéal pour les dictionnaires.

Exemple : Alias de Type (`type`)

Un alias de type permet de donner un nom plus significatif à un type complexe ou à une combinaison de types existants, améliorant ainsi la lisibilité et la maintenabilité de votre code.

```
// Définition d'un alias de type
type UserID = string | number; // Soit string, soit number (mais rien d'autre !)

// Utilisation de l'alias
const identifiant1: UserID = "JeanDupont"; // Valide (string)
const identifiant2: UserID = 42;           // Valide (number)
const identifiant3: UserID = { id: 42 };   // Invalide (object)
```

Exemple : Enum (Énumération)

Les énumérations vous permettent de définir un ensemble de constantes nommées, rendant votre code plus lisible et moins sujet aux erreurs lorsque vous utilisez des valeurs spécifiques qui ont un sens logique.

```
// Définition de l'énumération pour les directions de mouvement
enum Direction {
  Up = "W",
  Down = "S",
  Left = "A",
  Right = "D",
}

// Utilisation de l'énumération
function moveCharacter(direction: Direction): void {
  console.log(`Le personnage se déplace vers ${direction}`);
}

moveCharacter(Direction.Up);    // Affiche "Le personnage se déplace vers W"
moveCharacter(Direction.Left);  // Affiche "Le personnage se déplace vers A"

// Erreur : 'Z' n'est pas une direction valide selon notre énumération
// moveCharacter("Z"); // Argument de type 'string' non attribuable au paramètre de type 'Direction'.
```

À la compilation, l'enum devient un objet physique (conservé et exploitable) contrairement aux types.

Exemple : Interface (`interface`)

Une interface définit la "forme" d'un objet en spécifiant ses propriétés et leurs types. Elle garantit la cohérence des structures de données et facilite la collaboration en définissant des contrats clairs pour les objets.

```
// Définition de l'interface User
interface User {
  id: number;
  name: string;
  email: string;
  isAdmin?: boolean; // Propriété optionnelle
  roles: string[];
}

// Utilisation de l'interface
const user1: User = {
  id: 1,
  name: "Alice Dupont",
  email: "alice.dupont@example.com",
  roles: ["editor"]
};

const user2: User = {
  id: 2,
  name: "Bob Martin",
  email: "bob.martin@example.com",
  isAdmin: true, // La propriété optionnelle est présente ici
  roles: ["admin", "viewer"]
};
```

Exemple : Record (`Record`)

En TypeScript, le type utilitaire `Record` est utilisé pour construire un type d'objet dont les clés de propriétés sont d'un type `Keys` et dont les valeurs de propriétés sont d'un type `Type`. Il est particulièrement utile pour créer des dictionnaires ou des cartes où toutes les clés et toutes les valeurs suivent une structure spécifique.

```
// Définition de l'alias de type pour les catégories de produits
type ProductCategory = "Électronique" | "Vêtements" | "Livres";

// Utilisation de Record pour définir un catalogue de produits
// Les clés sont de type ProductCategory, et les valeurs sont de type number (quantités en stock)
const stock: Record<ProductCategory, number> = {
  Électronique: 150,
  Vêtements: 300,
  Livres: 200,
};
```

Typage de fonctions

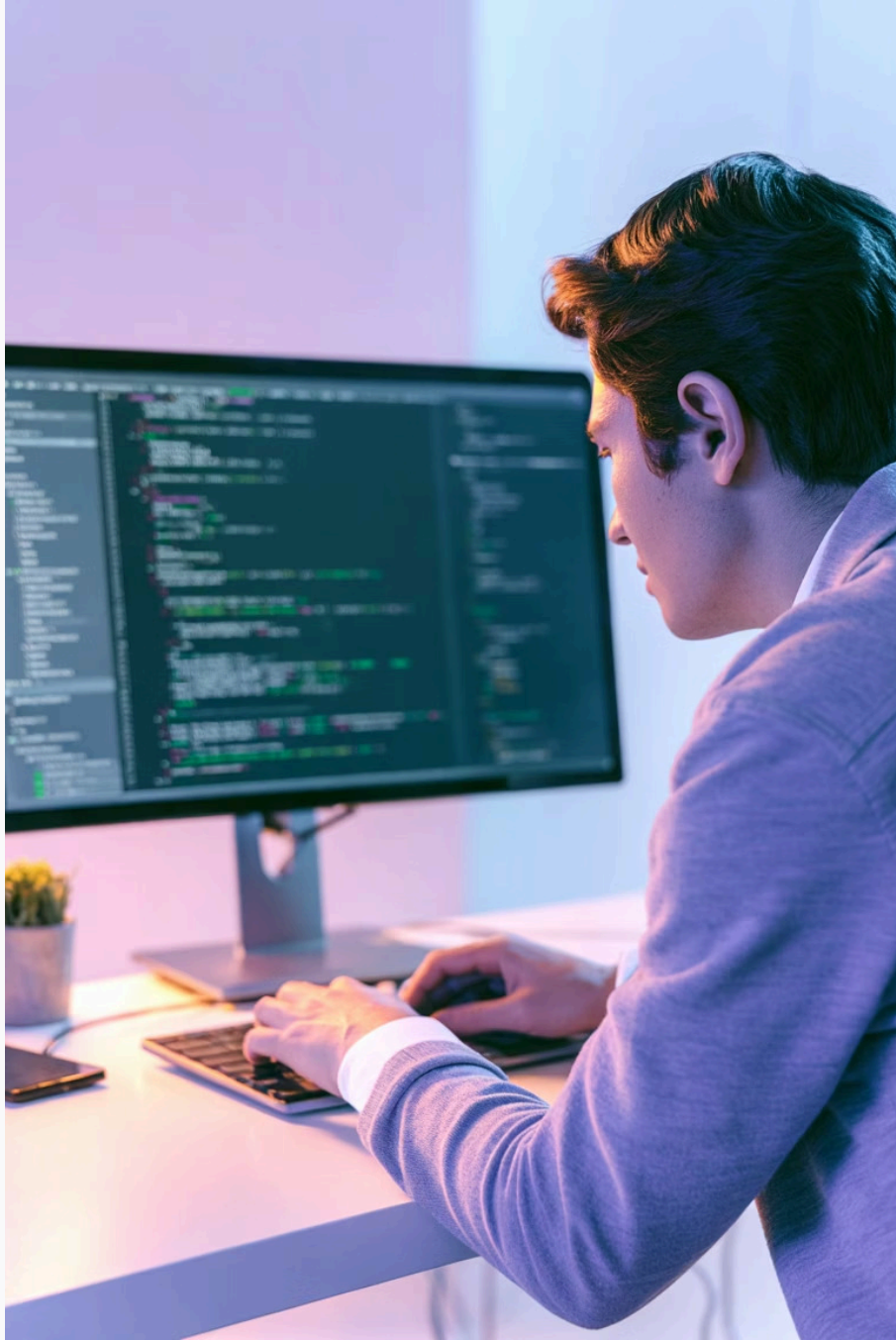
```
// Signature de fonction complète en TypeScript
function calculerPrixFinal(
  prixInitial: number,            // Paramètre typé 'number'
  tauxTaxe: number = 0.20,        // Paramètre typé 'number' avec valeur par défaut
  remise?: number                 // Paramètre optionnel typé 'number'
): number {                       // Type de retour 'number'
  let prixAvecTaxe = prixInitial * (1 + tauxTaxe);
  if (remise) prixAvecTaxe -= remise;

  return prixAvecTaxe;
}

// Exemples d'utilisation :
console.log(calculerPrixFinal(100));          // Affiche 120 (tauxTaxe par défaut)
console.log(calculerPrixFinal(100, 0.05));    // Affiche 105 (tauxTaxe spécifié)
console.log(calculerPrixFinal(100, 0.20, 10));  // Affiche 110 (tauxTaxe et remise spécifiés)
```

Trêve de blabla, place à un peu de pratique !

La théorie c'est sympa 30 secondes, mais c'est tout.



Mise en pratique : Typer la classe Task

À partir de la classe `Task` que nous avons vue précédemment, votre mission est de la typer entièrement avec TypeScript. Inspirez-vous de la JSDoc fournie pour déterminer les types corrects pour chaque propriété et paramètre.

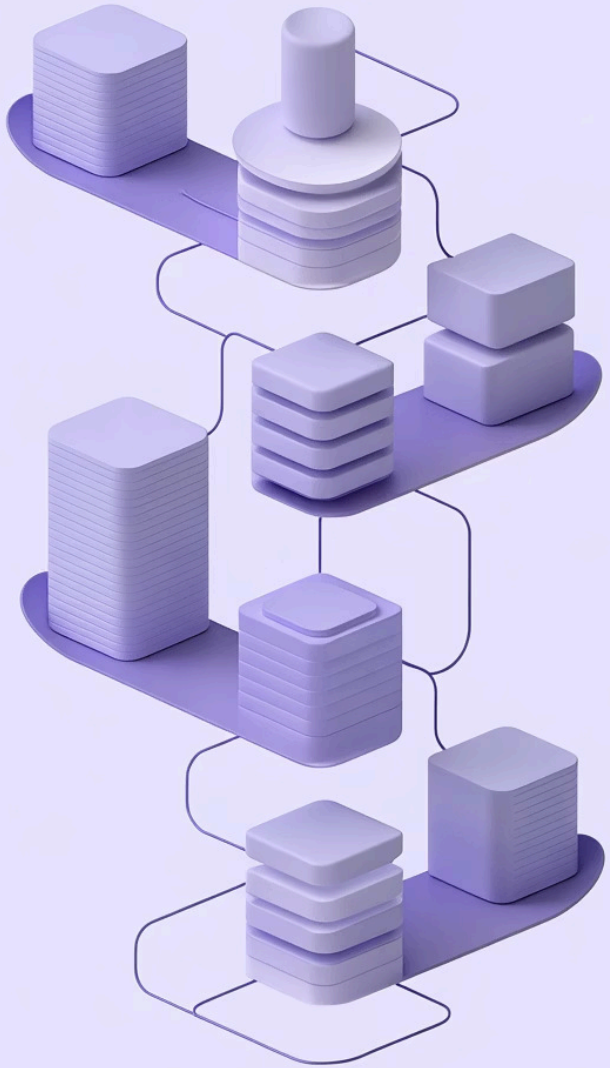
N'ayez pas peur du rouge : même si ça peut paraître frustrant, c'est lui qui vous assure de bien faire.

Bonus : Essayez de typer la classe TaskManager !

Fin de l'introduction à TypeScript

Et c'est déjà tout pour notre première incursion dans le monde de TypeScript !

Ce sera approfondi pendant votre deuxième année chez Coda, avec des concepts plus avancés. Pour l'instant, restons-en à une découverte 😉



MODULE

Introduction à la Canvas API

Introduction à la Canvas API

L'API Canvas vous offre un espace de dessin dynamique directement dans votre navigateur. Elle permet de générer des graphiques, animations et jeux en temps réel, offrant une flexibilité incroyable pour les expériences interactives.

Pour cette introduction, nous allons nous concentrer sur les éléments fondamentaux :

- L'élément HTML5 `<canvas>` comme base.
- Le contexte de rendu 2D pour nos opérations de dessin.
- La compréhension du système de coordonnées pour positionner vos formes et images.

Concepts clés de la Canvas API

1

L'élément HTML

Votre zone de dessin sur la page web. Il est vide par défaut et nécessite JavaScript pour afficher du contenu. Les attributs `width` et `height` définissent ses dimensions en pixels.

```
<canvas id="monCanvas" width="800" height="600"></canvas>
```

2

Récupération du contexte

Une fois l'élément `<canvas>` sélectionné, vous devez obtenir son "contexte de rendu" pour pouvoir dessiner. Pour le 2D, vous utilisez `getContext('2d')`.

```
const canvas = document.getElementById('monCanvas');
const ctx = canvas.getContext('2d');
```

3

La grille de dessin

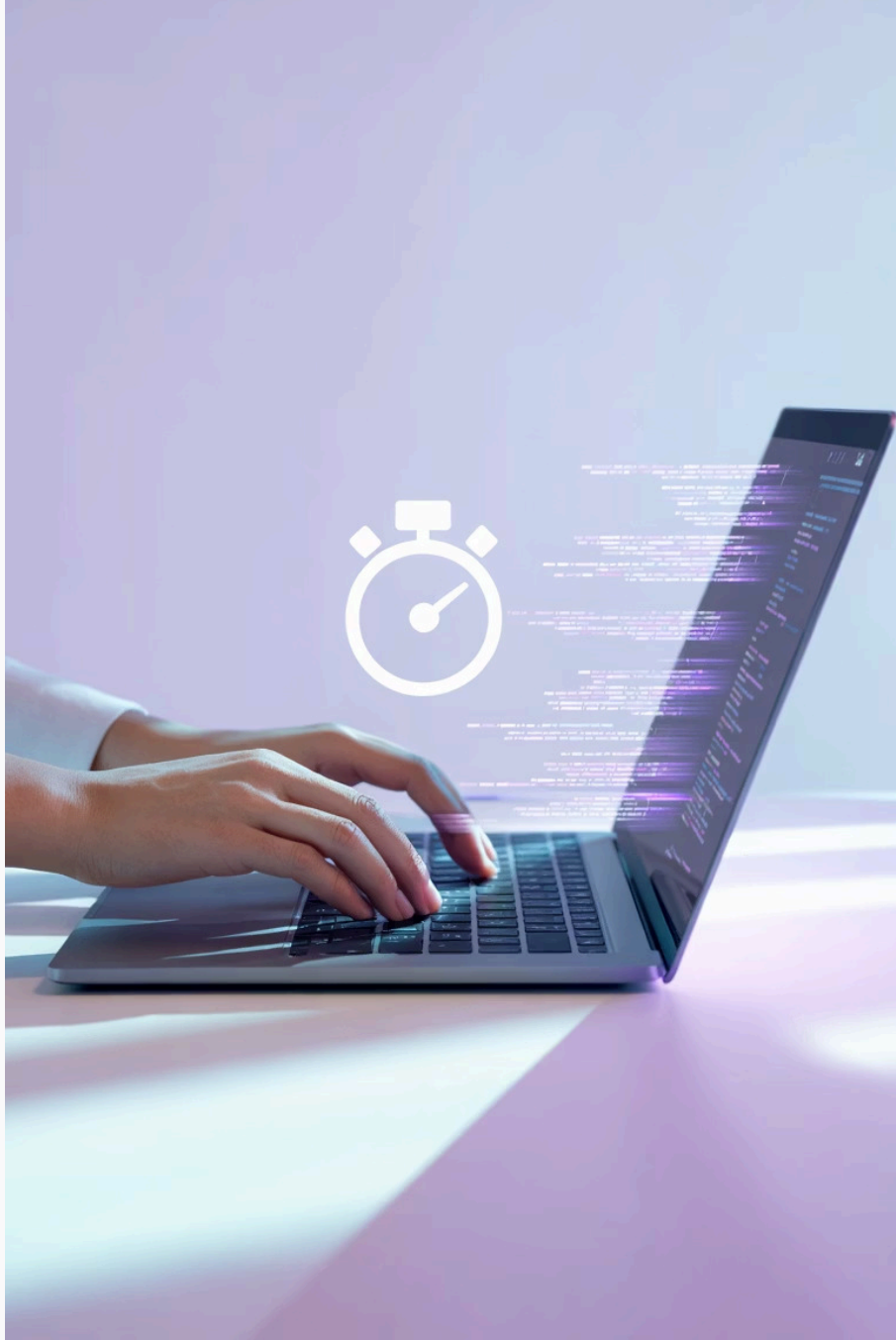
Le `<canvas>` utilise un système de coordonnées cartésiennes. Le point `(0, 0)` est toujours le coin supérieur gauche. L'axe X augmente vers la droite, et l'axe Y augmente vers le bas.

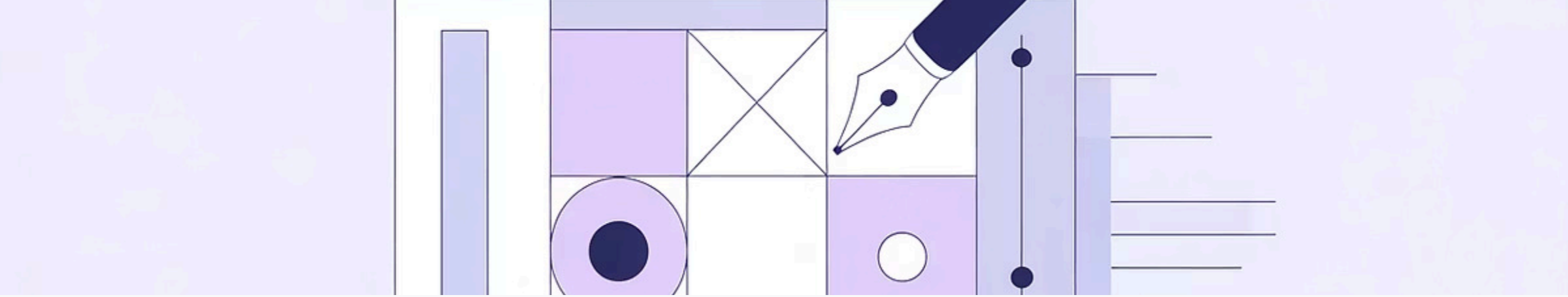
Mise en pratique : Initialisation du Canvas

Il est temps de passer à l'action ! Vous avez **5 minutes** pour :

1. Créer un élément `<canvas>` dans votre fichier HTML.
2. Le récupérer dans votre fichier TypeScript.
3. Récupérer son contexte de rendu 2D et le stocker dans une constante.

Indice : Les étapes sont très similaires à ce que nous avons vu sur la slide précédente.





Dessinons nos premiers carrés

Dans un premier temps, nous allons vouloir afficher quelque chose dans notre canvas.

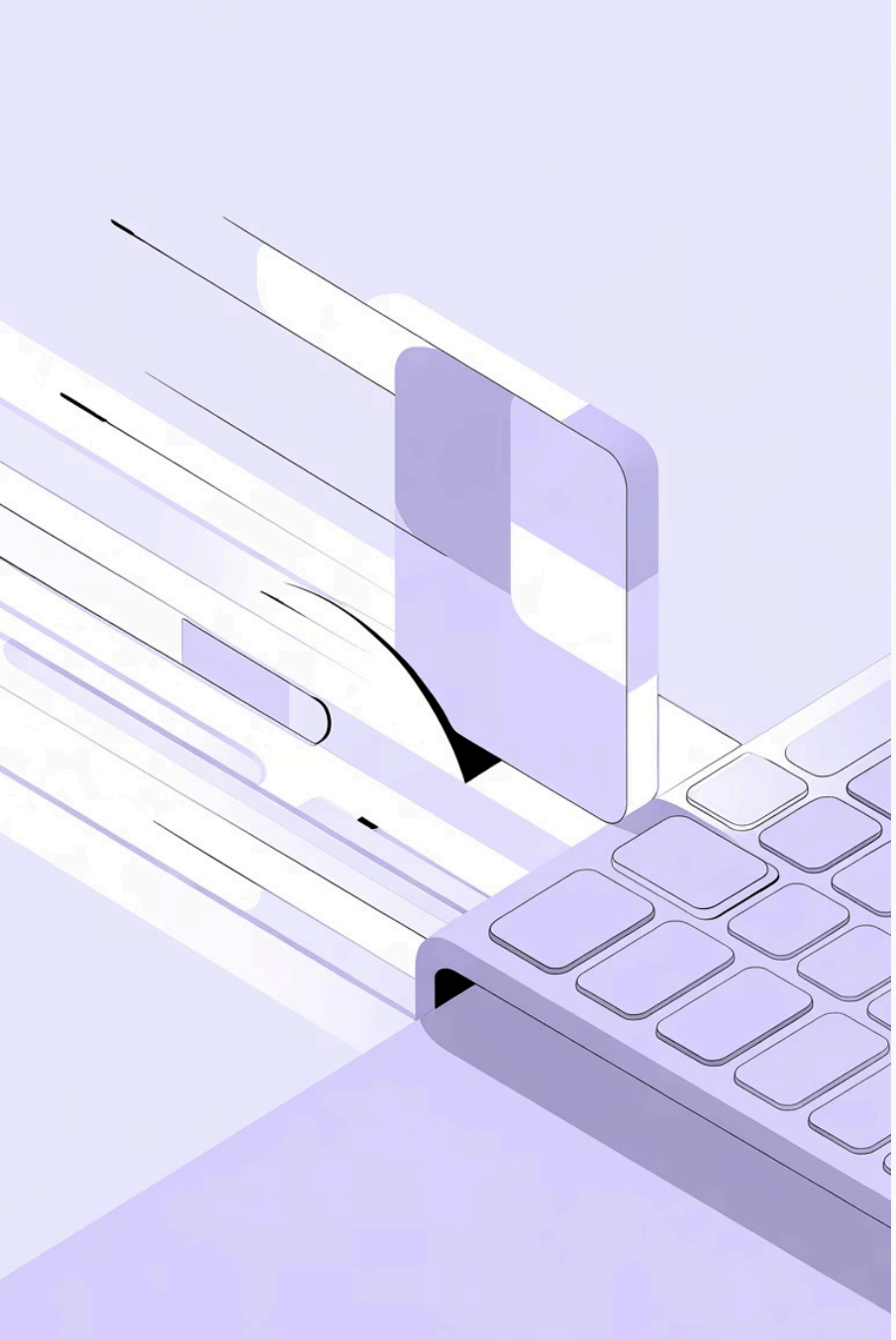
Ça ne va pas être beau, mais c'est la première étape indispensable pour réaliser quelque chose de topissime.
Nous commencerons par des formes simples pour maîtriser les bases du dessin sur le web.

Dessiner nos premières formes : Couleurs et contours

Utilisez `fillStyle` pour définir la couleur de remplissage et `fillRect(x, y, largeur, hauteur)` pour dessiner un rectangle plein. Par exemple, pour un carré rouge.

```
const canvas = document.getElementById('monCanvas') as HTMLCanvasElement;
const ctx = canvas.getContext('2d')!;

// Carré rouge plein
ctx.fillStyle = 'purple';
ctx.fillRect(50, 50, 100, 100);
```



Animer le carré : Interaction clavier

Avoir un carré statique, c'est un bon début, mais l'objectif est de le rendre interactif ! Nous allons maintenant explorer comment animer notre carré et le faire réagir aux actions de l'utilisateur.

Par exemple, nous pourrions lui permettre de se déplacer sur le canevas en réponse aux pressions sur les touches du clavier. Cela ouvre la porte à des expériences plus dynamiques et engageantes.

Animer le carré : Gestion des interactions clavier

Pour faire bouger notre carré, nous devons écouter les événements du clavier. Le code suivant met en place un système simple où les flèches directionnelles mettent à jour la position du carré à chaque image de l'animation.

```
import "./style.css";

const canvas = document.getElementById("app") as HTMLCanvasElement;
const ctx = canvas.getContext("2d")!;

let squareX = 50;
let squareY = 50;
const squareSize = 50;
const speed = 5;

canvas.width = window.innerWidth;
canvas.height = window.innerHeight;

const keysPressed: Record<string, boolean> = {};

document.addEventListener("keydown", (e: KeyboardEvent) => {
  keysPressed[e.key] = true;
});

document.addEventListener("keyup", (e: KeyboardEvent) => {
  keysPressed[e.key] = false;
});

function drawSquare() {
  ctx.fillStyle = "purple";
  ctx.fillRect(squareX, squareY, squareSize, squareSize);
}

function moveSquare() {
  if (keysPressed.ArrowUp) squareY -= speed;
  if (keysPressed.ArrowDown) squareY += speed;
  if (keysPressed.ArrowLeft) squareX -= speed;
  if (keysPressed.ArrowRight) squareX += speed;
}

function updateGame() {
  drawSquare();
  moveSquare();

  requestAnimationFrame(updateGame);
}

updateGame();
```

Question ouverte ?

Quels problèmes avez-vous rencontrés avec le code donné pour l'animation du carré ?

Résoudre le problème du tracé persistant

Pour résoudre le problème du carré qui conserve visuellement son ancienne position (ce qui crée un tracé), on va devoir **effacer** tout le canvas avant chaque nouveau dessin pour qu'il soit vide. Cela garantit que seule la position actuelle du carré est affichée.

```
function updateGame() {
  // Effacer tout le contenu du canvas avant de redessiner
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  // ...
}
```

Maintenir le carré dans les limites

Pour éviter que notre carré ne sorte de la zone visible du canvas, nous devons ajouter des vérifications de limites après chaque mise à jour de sa position. Ces contrôles garantissent que les coordonnées `squareX` et `squareY` restent toujours à l'intérieur des dimensions du canvas, quelle que soit la vitesse ou la direction du mouvement.

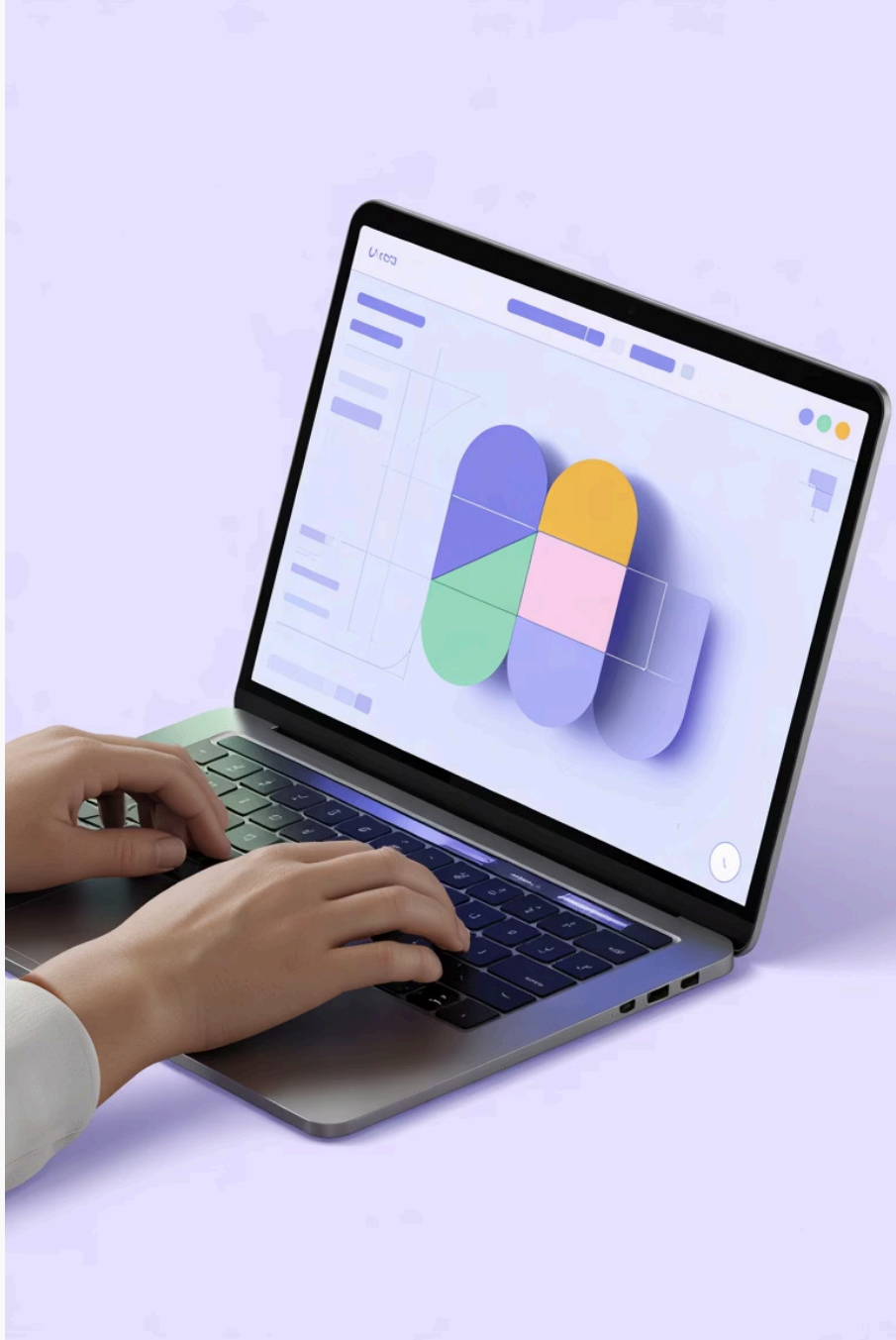
```
function moveSquare() {
  // ...

  squareX = Math.max(0, Math.min(squareX, canvas.width - squareSize));
  squareY = Math.max(0, Math.min(squareY, canvas.height - squareSize));
}
```

Mise en pratique autonome

Maintenant que notre carré est mobile, enrichissons notre expérience interactive ! Cette fois-ci, l'objectif est d'ajouter une fonctionnalité permettant de marquer la position du carré à la demande.

1. En appuyant sur la touche "Espace", dessinez un rond plein à la position actuelle du carré.
2. Assurez-vous que chaque rond dessiné reste visible même après que le canvas ait été effacé pour la prochaine frame. (Indice : Pensez à stocker ces ronds dans une structure de données !)
3. **Bonus :** Attribuez une couleur aléatoire à chaque nouveau rond parmi une liste prédéfinie.



Le Canvas API : Votre terrain de jeu créatif

Avec quelques lignes de code seulement, le Canvas API vous ouvre les portes de la création visuelle interactive. Des animations les plus folles aux jeux simples, tout est possible. C'est à vous de laisser libre cours à votre imagination pour créer l'inutile… et l'indispensable !

Avant de clôturer cette introduction, explorons ensemble quelques-unes des possibilités plus avancées et des usages élégants de cette puissante API.

Les bibliothèques qui exploitent Canvas API

La puissance de l'API Canvas se révèle pleinement à travers les bibliothèques qui l'exploitent. Elles transforment les fonctionnalités de base en outils de développement avancés, rendant la création visuelle accessible.

Graphiques et visualisation de données

- Chart.js (pour les graphiques interactifs)
- Konva.js (pour les scènes 2D complexes)
- P5.js (pour l'art et le design interactif)

Jeux et animations avancées

- Phaser (framework de jeux 2D)
- PixiJS (moteur de rendu 2D rapide)
- Three.js (pour la 3D sur le web, souvent combiné au Canvas)

À partir de la version MV, RPG Maker utilise PixiJS pour rendre tout le jeu, **100% JS** !

Exemples de jeux utilisant Canvas

L'API Canvas n'est pas seulement un outil puissant pour la visualisation de données ou les animations interactives ; elle est aussi la colonne vertébrale de nombreux jeux web, des plus simples aux plus complexes.

- **Vampire Survivors** (jeu indépendant développé par Poncle)
- **Slither.io, Agar.io, Diep.io, Krunker.io** (la plupart des jeux ".io" utilisent Canvas pour leur rendu)
- **QWOP**
- **Cookie Clicker**

Vous vous souvenez du premier jour, avec le site de Bruno où l'on contrôle une voiture ?

Eh bien c'est tout pareil : **Canvas** 😎

CONCLUSION

Et… on s'arrête là pour les découvertes du jour !

Il est temps de parler un peu d'une chose concrète : le projet final JavaScript.